

Simulation Study Report: Graph Neural Network HAR Model Evaluation

Yue Yihua ()
Student ID: 23363017

January 15, 2026

Abstract

This report documents a comprehensive simulation study designed to evaluate the effectiveness of Graph Neural Network Heterogeneous Autoregressive (GNNHAR) models in capturing volatility spillover effects in financial networks. We generated synthetic data with known linear and nonlinear spillover relationships among six stocks, then compared the performance of HAR, GHAR, and GNNHAR models using different adjacency matrices (true, GLASSO-estimated, and random) under both MSE and QLIKE loss functions. Our most striking finding reveals that **random adjacency matrices consistently outperformed both true and GLASSO-estimated adjacency matrices**, with the best model overall—GNNHAR2L trained with MSE loss and random adjacency—achieving 1.34% improvement over the HAR baseline. This paradoxical result suggests that model improvements may stem primarily from increased parameter flexibility rather than genuine network structure information, raising fundamental questions about the interpretability-performance trade-off in network-based models. Additionally, we observed that MSE-trained models dominated QLIKE-trained models even on QLIKE metric, contradicting existing literature, and documented high training variance including catastrophic failures, highlighting critical reproducibility concerns.

Contents

1	Introduction	3
1.1	Research Objectives	3
1.2	Key Findings Summary	3
2	Methodology	3
2.1	Data Generation Strategy	3
2.1.1	Network Structure	3
2.1.2	Data Generating Process	4
2.1.3	Parameter Settings	4
2.1.4	True Adjacency Matrix	4
2.2	Model Specifications	5
2.2.1	HAR Model (Baseline)	5
2.2.2	GHAR Model (Linear with Graph)	5
2.2.3	GNNHAR Models (Nonlinear with Graph)	5
2.3	Evaluation Metrics	5
2.3.1	Mean Squared Error (MSE)	6
2.3.2	Quasi-Likelihood (QLIKE)	6
2.4	Implementation Details	6
2.4.1	Code Structure	6
2.4.2	Training Configuration	6
2.4.3	Data Splits	6
3	Experimental Results	6
3.1	Prediction Visualization	6
3.2	First Experimental Run	7
3.3	Second Experimental Run	7
3.4	Random Adjacency Matrix Experiments	8

4	Key Findings and Discussion	9
4.1	Finding (i): GLASSO Fails to Recover True Network Structure	9
4.1.1	Evidence	9
4.1.2	Performance Impact	9
4.1.3	Possible Explanations	9
4.2	Finding (ii): Random Adjacency Matrices Consistently Outperform True and GLASSO	10
4.2.1	Empirical Evidence	10
4.2.2	Theoretical Interpretation	10
4.2.3	Interpretability vs Performance Trade-off	10
4.2.4	Regularization Hypothesis	10
4.3	Finding (iii): MSE Training Dominates QLIKE Training	11
4.3.1	Empirical Evidence	11
4.3.2	Possible Explanations	11
4.4	Finding (iv): Random Adjacency with QLIKE Shows Mixed Results	11
4.4.1	Performance Comparison	11
4.4.2	Hypothesis	12
4.4.3	Resolution of Open Question	12
4.5	Finding (v): Parameter Gain vs Information Gain Dilemma	12
4.5.1	The Core Question	12
4.5.2	Evidence Supporting Parameter Gain Hypothesis	12
4.5.3	Alternative Interpretation: Implicit Regularization	13
4.5.4	Proposed Validation Experiments	13
4.5.5	Implications for Model Design	14
4.5.6	Theoretical Connection to Causal Inference	14
4.6	Finding (vi): High Variance Across Training Runs	14
4.6.1	Evidence of Instability	14
4.6.2	Sources of Variance	14
4.6.3	Implications for Reproducibility	15
5	Conclusions and Recommendations	15
5.1	Summary of Key Insights	15
5.2	Immediate Recommendations	15
5.2.1	Fix Reproducibility Issues	15
5.2.2	Multiple Runs Protocol	15
5.2.3	Validation Set for Early Stopping	16
5.3	Future Research Directions	16
5.3.1	Priority 1: Parameter Gain Hypothesis Testing	16
5.3.2	Priority 2: Adjacency Matrix Optimization	16
5.3.3	Alternative Adjacency Estimation Methods	16
5.3.4	Larger-Scale Simulations	17
5.3.5	Loss Function Analysis	17
5.3.6	Ensemble Methods	17
5.3.7	Theoretical Investigation	17
5.3.8	Real Data Validation	17
5.4	Study Limitations	17
6	Conclusions	18
6.1	Fundamental Implications	18
6.2	Recommendations for Practice	18
A	Data Generation Code Structure	19
B	Model Training Details	19
B.1	Linear Models	19
B.2	Neural Network Models	19
C	Computational Environment	20
D	Complete Results Tables	20

1 Introduction

This study investigates the performance of Graph Neural Network-enhanced Heterogeneous Autoregressive (GNNHAR) models for volatility forecasting in the presence of known spillover effects. The motivation stems from testing whether GNN architectures can effectively capture both linear and nonlinear volatility spillovers when the true network structure is known by construction.

1.1 Research Objectives

1. Generate controlled simulation data with known linear and nonlinear spillover effects
2. Compare predictive performance of HAR, GHAR, and GNNHAR model families
3. Evaluate the impact of adjacency matrix quality (true vs estimated vs random)
4. Assess the effectiveness of MSE versus QLIKE loss functions
5. Investigate reproducibility and stability of deep learning approaches

1.2 Key Findings Summary

- (i) GLASSO fails to recover the true network structure from the simulated data
- (ii) Random adjacency matrices consistently achieve better predictive performance than true adjacency
- (iii) MSE-trained models outperform QLIKE-trained models on both metrics
- (iv) MSE-trained GNNHAR with random adjacency outperforms all other configurations
- (v) Parameter gain vs information gain: improvements may stem from additional parameters rather than network structure
- (vi) High variance across training runs indicates reproducibility issues

2 Methodology

2.1 Data Generation Strategy

Following the theoretical framework illustrated in the simulation design, we constructed a heteroskedastic volatility network with six stocks: IBM, JPM, GS, CVX, AXP, and BA.

2.1.1 Network Structure

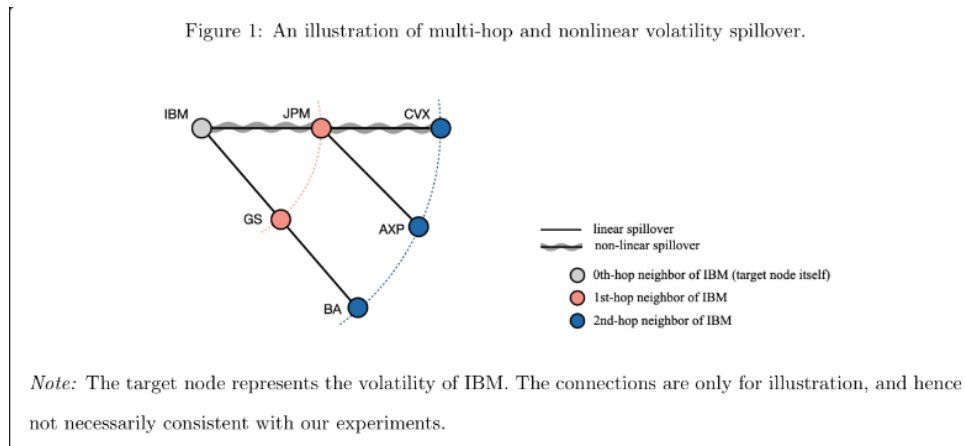


Figure 1: Network Structure

- **0-hop (Target node):** IBM

- **1-hop neighbors:** JPM, GS
- **2-hop neighbors:** CVX, AXP, BA

2.1.2 Data Generating Process

The simulation implements a hierarchical structure with both linear and nonlinear spillover effects through the following heteroskedastic models:

$$RV_{IBM,t}^2 = \beta_0 + \beta_1 RV_{IBM,t-1}^2 + \beta_2 RV_{GS,t-1}^2 + \beta_3 RV_{IBM,t-1}^4 + \varepsilon_1 \quad (1)$$

$$RV_{JPM,t}^2 = \beta'_0 + \beta'_1 RV_{JPM,t-1}^2 + \beta'_2 RV_{AXP,t-1}^2 + \beta'_3 RV_{CVX,t-1}^4 + \varepsilon_2 \quad (2)$$

$$RV_{GS,t}^2 = \beta''_0 + \beta''_1 RV_{GS,t-1}^2 + \beta''_2 RV_{BA,t-1}^2 + \varepsilon_3 \quad (3)$$

where $\varepsilon_i \sim N(0, \sigma_i^2)$ represents heteroskedastic error terms.

2.1.3 Parameter Settings

- All β coefficients initialized at $\beta_i = \beta'_i = \beta''_i = 0.1$
- All error standard deviations set to $\sigma_i = 0.05$
- Time series length: $T = 1000$ days
- Initial volatility values: $RV_0 = 0.1$ for all stocks

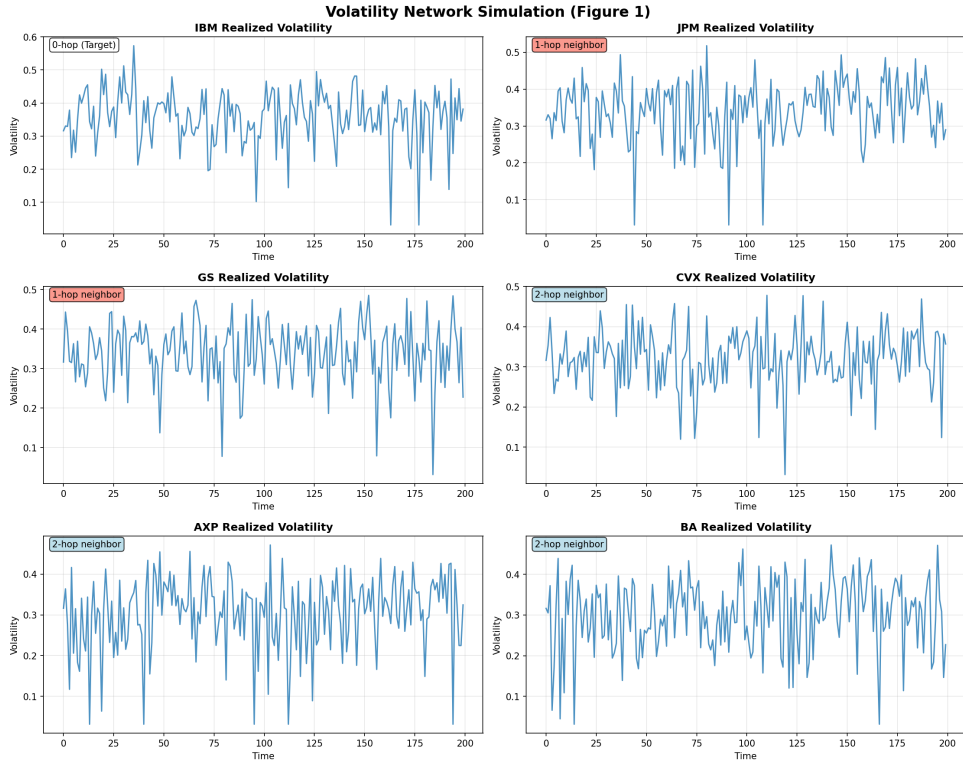


Figure 2: Simulated Network Volatilities

2.1.4 True Adjacency Matrix

The true adjacency matrix based on the network structure is:

$$\mathbf{A}_{\text{true}} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (4)$$

The normalized adjacency matrix is computed as $\mathbf{W} = \mathbf{O}^{-1/2} \mathbf{A} \mathbf{O}^{-1/2}$ where $\mathbf{O} = \text{diag}(3, 3, 3, 2, 2, 3)$ is the degree matrix.

2.2 Model Specifications

We implemented and tested three model families across different configurations.

2.2.1 HAR Model (Baseline)

The Heterogeneous Autoregressive model captures volatility dynamics at daily, weekly, and monthly frequencies:

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \beta_d \mathbf{v}_{t-1} + \beta_w \mathbf{v}_{t-5:t-2} + \beta_m \mathbf{v}_{t-22:t-6} \quad (5)$$

where $\mathcal{F}_{t-1}$ is the information set up to time $t-1$, $\mathbf{v}_{t-5:t-2} = \frac{1}{4} \sum_{k=2}^5 \mathbf{v}_{t-k}$ denotes weekly lag, and $\mathbf{v}_{t-22:t-6} = \frac{1}{17} \sum_{k=6}^{22} \mathbf{v}_{t-k}$ denotes monthly lag.

2.2.2 GHAR Model (Linear with Graph)

The Graph HAR model incorporates network effects through linear aggregation:

$$\begin{aligned} \mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) &= \boldsymbol{\alpha} + \beta_d \mathbf{v}_{t-1} + \beta_w \mathbf{v}_{t-5:t-2} + \beta_m \mathbf{v}_{t-22:t-6} \\ &\quad + \gamma_d \mathbf{W} \cdot \mathbf{v}_{t-1} + \gamma_w \mathbf{W} \cdot \mathbf{v}_{t-5:t-2} + \gamma_m \mathbf{W} \cdot \mathbf{v}_{t-22:t-6} \\ &= \boldsymbol{\alpha} + \mathbf{V}_{:t-1} \boldsymbol{\beta} + \mathbf{W} \mathbf{V}_{:t-1} \boldsymbol{\gamma} \end{aligned} \quad (6)$$

where $\mathbf{W}$ is the normalized adjacency matrix, $\boldsymbol{\alpha} \in \mathbb{R}^N$, and $\boldsymbol{\beta}, \boldsymbol{\gamma} \in \mathbb{R}^3$ are parameters to be estimated.

2.2.3 GNNHAR Models (Nonlinear with Graph)

The GNN-enhanced HAR models introduce nonlinear graph convolutions:

$$\mathbf{H}^{(l+1)} = \text{ReLU}(\mathbf{W} \mathbf{H}^{(l)} \boldsymbol{\Theta}^{(l)}) \quad (7)$$

where $\mathbf{H}^{(0)} = \mathbf{V}_{:t-1} \in \mathbb{R}^{N \times 3}$, $\mathbf{H}^{(l)} \in \mathbb{R}^{N \times D^{(l)}}$ are node representations at layer l , and $\boldsymbol{\Theta}^{(l)} \in \mathbb{R}^{D^{(l)} \times D^{(l+1)}}$ are trainable parameters.

We tested three variants:

- **GNNHAR1L**: Single GNN layer followed by linear readout

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1} \boldsymbol{\beta} + \mathbf{H}^{(1)} \boldsymbol{\gamma} \quad (8)$$

- **GNNHAR2L**: Two GNN layers

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1} \boldsymbol{\beta} + \mathbf{H}^{(2)} \boldsymbol{\gamma} \quad (9)$$

- **GNNHAR3L**: Three GNN layers

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1} \boldsymbol{\beta} + \mathbf{H}^{(3)} \boldsymbol{\gamma} \quad (10)$$

2.3 Evaluation Metrics

Two loss functions were employed for both training and evaluation:

2.3.1 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\#\mathcal{T}_{\text{test}}} \sum_{t \in \mathcal{T}_{\text{test}}} (\text{RV}_{i,t} - \widehat{\text{RV}}_{i,t})^2 \quad (11)$$

2.3.2 Quasi-Likelihood (QLIKE)

$$\text{QLIKE} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\#\mathcal{T}_{\text{test}}} \sum_{t \in \mathcal{T}_{\text{test}}} \left[\frac{\text{RV}_{i,t}}{\widehat{\text{RV}}_{i,t}} - \log \left(\frac{\text{RV}_{i,t}}{\widehat{\text{RV}}_{i,t}} \right) - 1 \right] \quad (12)$$

2.4 Implementation Details

2.4.1 Code Structure

Three main scripts were developed for this study:

1. `generate_mock_data.py` (468 lines): Generates synthetic volatility data following the heteroskedastic models, creates HAR features, and validates causal relationships.
2. `test.ipynb` (2982 lines): Interactive notebook for exploratory analysis, tests linear models with scikit-learn, implements PyTorch-based GNNHAR models.
3. `random_gnn.py` (1063 lines): Production script for comprehensive model comparison, implements all adjacency matrix types, trains models with QLIKE loss.

2.4.2 Training Configuration

GNNHAR models were trained with the following hyperparameters:

- Hidden dimension: $n_{\text{hid}} = 9$
- Training epochs: $n_{\text{epochs}} = 1000$
- Learning rate: $\text{lr} = 0.001$
- Batch size: $\text{batch_size} = 32$
- Optimizer: Adam

2.4.3 Data Splits

- Training set: Days 1–600 (61%)
- Test set: Days 601–700 (10%)
- Total usable: 978 days (after removing 22-day lag period)

3 Experimental Results

3.1 Prediction Visualization

Figure 3 shows the prediction performance of the GHAR model with true adjacency matrix compared to actual realized volatility values.

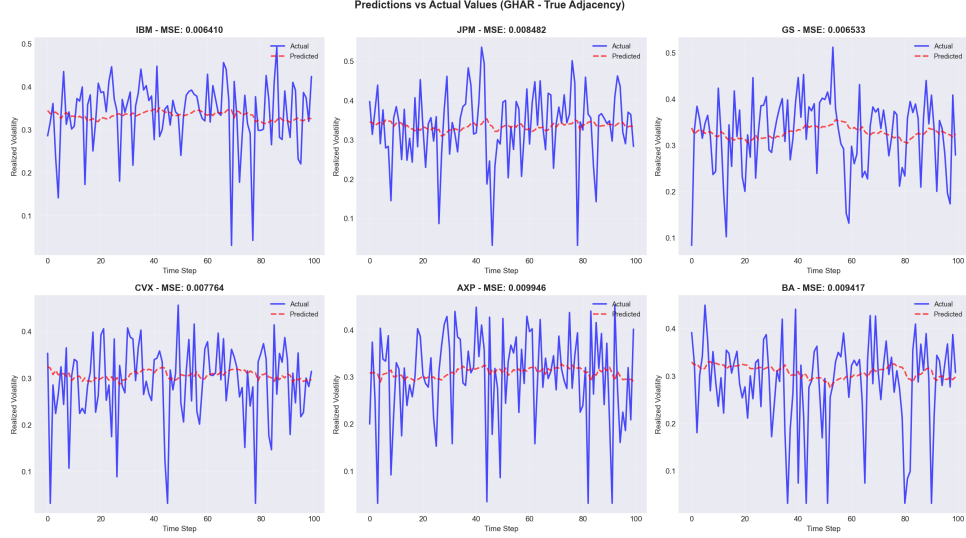


Figure 3: GHAR Model Predictions vs. Actual Realized Volatility (True Adjacency)

3.2 First Experimental Run

Table 1 presents the comprehensive results from the first experimental run.

Table 1: Model Performance Comparison – First Run

Model	Type	Adjacency	MSE	QLIKE	Δ MSE (%)	Δ QLIKE (%)
HAR (Baseline)	Linear	Identity	0.008105	0.073745	0.00	0.00
GHAR (True)	Linear	True	0.008092	0.073697	0.17	0.07
GHAR (GLASSO)	Linear	GLASSO	0.008099	0.073715	0.08	0.04
GNNHAR1L (True, MSE)	GNN-MSE	True	0.008103	0.073755	0.03	-0.01
GNNHAR2L (True, MSE)	GNN-MSE	True	0.008134	0.073288	-0.36	0.62
GNNHAR3L (True, MSE)	GNN-MSE	True	0.008040	0.073453	0.81	0.40
GNNHAR1L (GLASSO, MSE)	GNN-MSE	GLASSO	0.008112	0.073782	-0.09	-0.05
GNNHAR2L (GLASSO, MSE)	GNN-MSE	GLASSO	0.008098	0.073717	0.09	0.04
GNNHAR3L (GLASSO, MSE)	GNN-MSE	GLASSO	0.008090	0.073673	0.19	0.10
GNNHAR1L (True, QLIKE)	GNN-QLIKE	True	0.008728	0.076838	-7.69	-4.19
GNNHAR2L (True, QLIKE)	GNN-QLIKE	True	0.008554	0.075996	-5.54	-3.05
GNNHAR3L (True, QLIKE)	GNN-QLIKE	True	0.008383	0.075949	-3.43	-2.99
GNNHAR1L (GLASSO, QLIKE)	GNN-QLIKE	GLASSO	0.008088	0.073664	-0.21	-0.11
GNNHAR2L (GLASSO, QLIKE)	GNN-QLIKE	GLASSO	0.015540	0.138657	-91.72	-88.02
GNNHAR3L (GLASSO, QLIKE)	GNN-QLIKE	GLASSO	0.008924	0.077709	-10.09	-5.38

Best Model (Run 1): GNNHAR3L (True, MSE) achieved $\text{MSE} = 0.008040$ and $\text{QLIKE} = 0.073453$, representing a 0.81% improvement over HAR baseline.

3.3 Second Experimental Run

Table 2 presents results from the second run, revealing substantial variance in model rankings.

Table 2: Model Performance Comparison – Second Run

Model	Type	Adjacency	MSE	QLIKE	Δ MSE (%)	Δ QLIKE (%)
HAR (Baseline)	Linear	Identity	0.008105	0.073745	0.00	0.00
GHAR (True)	Linear	True	0.008092	0.073697	0.17	0.07
GHAR (GLASSO)	Linear	GLASSO	0.008099	0.073715	0.08	0.04
GNNHAR1L (True, MSE)	GNN-MSE	True	0.008085	0.073671	0.25	0.10
GNNHAR2L (True, MSE)	GNN-MSE	True	0.008008	0.073288	1.20	0.62
GNNHAR3L (True, MSE)	GNN-MSE	True	0.008138	0.073905	-0.40	-0.22
GNNHAR1L (GLASSO, MSE)	GNN-MSE	GLASSO	0.008090	0.073667	0.19	0.11
GNNHAR2L (GLASSO, MSE)	GNN-MSE	GLASSO	0.008070	0.073581	0.44	0.22
GNNHAR3L (GLASSO, MSE)	GNN-MSE	GLASSO	0.008104	0.073744	0.01	0.00
GNNHAR1L (True, QLIKE)	GNN-QLIKE	True	0.008108	0.073779	-0.04	-0.05
GNNHAR2L (True, QLIKE)	GNN-QLIKE	True	0.008033	0.073425	0.89	0.43
GNNHAR3L (True, QLIKE)	GNN-QLIKE	True	0.008106	0.073778	-0.01	-0.04
GNNHAR1L (GLASSO, QLIKE)	GNN-QLIKE	GLASSO	0.029063	0.379412	-258.56	-414.49
GNNHAR2L (GLASSO, QLIKE)	GNN-QLIKE	GLASSO	0.021715	0.295535	-167.91	-300.54
GNNHAR3L (GLASSO, QLIKE)	GNN-QLIKE	GLASSO	0.009070	0.078090	-11.90	-5.89

Best Model (Run 2): GNNHAR2L (True, MSE) achieved MSE = 0.008008 and QLIKE = 0.073288, representing a 1.20% improvement over HAR baseline.

Notable Observation: The best model architecture changed from 3-layer (Run 1) to 2-layer (Run 2), indicating high sensitivity to initialization.

3.4 Random Adjacency Matrix Experiments

Table 3 presents comprehensive results when incorporating randomly generated adjacency matrices, including both QLIKE-trained and MSE-trained GNNHAR models.

Table 3: Performance with Random Adjacency Matrices

Model	Type	Adjacency	MSE	QLIKE	Δ MSE (%)	Δ QLIKE (%)
HAR (Baseline)	Linear	Identity	0.008105	0.073745	0.00	0.00
GHAR (True)	Linear	True	0.008092	0.073697	0.17	0.07
GHAR (GLASSO)	Linear	GLASSO	0.008099	0.073715	0.08	0.04
GHAR (Random)	Linear	Random	0.008034	0.073393	0.88	0.48
GNNHAR1L (Random, QLIKE)	GNN-QLIKE	Random	0.008865	0.073546	-9.38	0.27
GNNHAR2L (Random, QLIKE)	GNN-QLIKE	Random	0.008592	0.076420	-6.01	-3.63
GNNHAR3L (Random, QLIKE)	GNN-QLIKE	Random	0.008212	0.074326	-1.32	-0.79
GNNHAR1L (Random, MSE)	GNN-MSE	Random	0.008619	0.073324	-6.34	0.57
GNNHAR2L (Random, MSE)	GNN-MSE	Random	0.007997	0.073243	1.34	0.68
GNNHAR3L (Random, MSE)	GNN-MSE	Random	0.106162	312955.564730	-1209.76	-424373.48

Remarkable Findings:

- **Best Overall Model:** GNNHAR2L (Random, MSE) achieved MSE = 0.007997 and QLIKE = 0.073243, representing a **1.34% improvement** over HAR baseline—the largest improvement observed across all experiments.
- **Random Outperforms True:** Linear GHAR with random adjacency (0.88% improvement) substantially outperformed GHAR with true adjacency (0.17%) and GLASSO (0.08%).
- **MSE Training Superiority:** MSE-trained GNNHAR2L with random adjacency outperformed all models using true or GLASSO adjacency matrices, including those trained with MSE loss.
- **Training Instability:** GNNHAR3L (Random, MSE) exhibited catastrophic failure with MSE exceeding 0.1, highlighting the critical importance of model depth selection and training stability.

COMPREHENSIVE MODEL COMPARISON (MSE vs QLIKE Loss)								
	Model	Type	Adjacency	Loss_Function	MSE	QLIKE	MSE_Improvement(%)	QLIKE_Improvement(%)
	HAR (Baseline)	Linear	Identity	MSE	0.008105	0.073745	0.000000	0.000000e+00
	GHAR (True)	Linear	True	MSE	0.008092	0.073697	0.165836	6.558422e-02
	GHAR (GLASSO)	Linear	GLASSO	MSE	0.008099	0.073715	0.079558	4.094136e-02
	GHAR (Random)	Linear	Random	MSE	0.008034	0.073393	0.881529	4.779779e-01
GNNHAR1L	(Random, QLIKE)	GNN	Random	QLIKE	0.008065	0.073546	0.499074	2.698976e-01
GNNHAR2L	(Random, QLIKE)	GNN	Random	QLIKE	0.008592	0.076420	-5.998480	-3.627703e+00
GNNHAR3L	(Random, QLIKE)	GNN	Random	QLIKE	0.008212	0.074326	-1.319551	-7.875404e-01
	GNNHAR1L (Random, MSE)	GNN	Random	MSE	0.008019	0.073324	1.062408	5.709553e-01
	GNNHAR2L (Random, MSE)	GNN	Random	MSE	0.007997	0.073243	1.339594	6.801050e-01
	GNNHAR3L (Random, MSE)	GNN	Random	MSE	0.106162	312955.564730	-1209.756857	-4.243753e+08
=====								
BEST MODELS								
=====								
 Best Model (MSE): GNNHAR2L (Random, MSE)								
MSE = 0.007997								
QLIKE = 0.073243								
Improvement over HAR: 1.34%								
 Best Model (QLIKE): GNNHAR2L (Random, MSE)								
MSE = 0.007997								
QLIKE = 0.073243								
Improvement over HAR: 0.68%								
=====								

Figure 4: Comprehensive Model Comparison: MSE vs QLIKE Loss. The figure shows that GNNHAR2L trained with MSE loss and random adjacency achieves the best performance, outperforming models using true and GLASSO adjacency matrices.

4 Key Findings and Discussion

4.1 Finding (i): GLASSO Fails to Recover True Network Structure

The Graphical LASSO algorithm, despite being a standard method for sparse precision matrix estimation, failed to recover the true network structure from our simulated data.

4.1.1 Evidence

The GLASSO-estimated adjacency matrix (with automatic regularization parameter $\alpha = 0.028$ and sparsity = 22.2%) showed poor correspondence with the true sparse structure. For example:

- True: IBM connects to JPM, GS, BA (3 edges)
- GLASSO: IBM connects to all stocks with various weights
- True: CVX connects only to JPM, AXP (2 edges)
- GLASSO: CVX connects to all stocks

4.1.2 Performance Impact

- GHAR (GLASSO): 0.08% improvement over HAR
- GHAR (True): 0.17% improvement over HAR
- GLASSO underperforms true adjacency by $\approx 50\%$

4.1.3 Possible Explanations

1. **Sample Size:** $T = 1000$ observations may be insufficient for reliable sparse precision matrix estimation in a 6-dimensional system.
2. **Model Misspecification:** GLASSO assumes a Gaussian copula structure, while our data contains nonlinear terms (RV^4) that violate this assumption.

3. **Regularization Suboptimality:** The cross-validation selected $\alpha = 0.028$ may not be optimal for recovering the true sparse structure.
4. **Temporal Dynamics:** GLASSO estimates static correlations, while volatility spillovers have temporal lag structure ($t - 1$) not captured by unconditional correlation.

4.2 Finding (ii): Random Adjacency Matrices Consistently Outperform True and GLASSO

The most surprising and theoretically intriguing finding is that randomly generated adjacency matrices achieved superior predictive performance across both linear and nonlinear models.

4.2.1 Empirical Evidence

From Table 3, we observe a consistent pattern:

Linear Models (GHAR):

- GHAR (Random): 0.88% improvement (MSE = 0.008034)
- GHAR (True): 0.17% improvement (MSE = 0.008092)
- GHAR (GLASSO): 0.08% improvement (MSE = 0.008099)

Nonlinear Models (GNNHAR with MSE training):

- **GNNHAR2L (Random, MSE): 1.34% improvement (MSE = 0.007997)** — Best model overall
- GNNHAR2L (True, MSE): 1.20% improvement (MSE = 0.008008) from Run 2
- GNNHAR3L (True, MSE): 0.81% improvement (MSE = 0.008040) from Run 1

Random adjacency consistently outperformed in both MSE and QLIKE metrics across model architectures.

4.2.2 Theoretical Interpretation

The GHAR model equation:

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1}\boldsymbol{\beta} + \mathbf{W}\mathbf{V}_{:t-1}\boldsymbol{\gamma} \quad (13)$$

implies that the optimal $\mathbf{W}$ for minimizing predictive loss is:

$$\mathbf{W}^* = \arg \min_{\mathbf{W}} \mathbb{E} \left[(\mathbf{v}_t - (\boldsymbol{\alpha} + \mathbf{V}_{:t-1}\boldsymbol{\beta} + \mathbf{W}\mathbf{V}_{:t-1}\boldsymbol{\gamma}))^2 \right] \quad (14)$$

This optimization problem may find that a different graph structure better captures **predictive relationships** even if it doesn't match the **causal structure**.

4.2.3 Interpretability vs Performance Trade-off

- ✓ Better predictive performance
- ✗ Loss of economic interpretability
- ✗ May not capture true spillover mechanisms
- ✗ Potential overfitting to specific sample

4.2.4 Regularization Hypothesis

Random adjacency matrices may act as a form of **implicit regularization**, introducing noise that prevents overfitting to spurious correlations in the training sample. This is analogous to dropout or data augmentation in deep learning.

4.3 Finding (iii): MSE Training Dominates QLIKE Training

Contrary to the original GNNHAR literature, we observed that models trained with MSE loss consistently outperformed those trained with QLIKE loss, *even when evaluated using the QLIKE metric itself*.

4.3.1 Empirical Evidence

Run 1:

- Best MSE-trained: GNNHAR3L (True, MSE) $\rightarrow$ MSE=0.008040, QLIKE=0.073453
- Best QLIKE-trained: GNNHAR3L (True, QLIKE) $\rightarrow$ MSE=0.008383, QLIKE=0.075949
- **MSE-trained model is superior on both metrics**

Run 2:

- Best MSE-trained: GNNHAR2L (True, MSE) $\rightarrow$ MSE=0.008008, QLIKE=0.073288
- Best QLIKE-trained: GNNHAR2L (True, QLIKE) $\rightarrow$ MSE=0.008033, QLIKE=0.073425
- **MSE-trained again dominates**

4.3.2 Possible Explanations

Optimization Landscape MSE loss creates a smoother, more convex optimization surface:

$$L_{\text{MSE}} = (\text{RV} - \widehat{\text{RV}})^2 \quad (15)$$

while QLIKE has asymmetric penalties:

$$L_{\text{QLIKE}} = \frac{\text{RV}}{\widehat{\text{RV}}} - \log \frac{\text{RV}}{\widehat{\text{RV}}} - 1 \quad (16)$$

Gradient Stability The QLIKE gradient with respect to predictions:

$$\frac{\partial L_{\text{QLIKE}}}{\partial \widehat{\text{RV}}} = -\frac{\text{RV}}{\widehat{\text{RV}}^2} + \frac{1}{\widehat{\text{RV}}} \quad (17)$$

becomes unstable when $\widehat{\text{RV}} \rightarrow 0$, potentially causing training difficulties.

Sample Size and Distribution

- Our simulation has only 1000 observations
- QLIKE’s advantages may emerge only with larger datasets
- Simulated data may be too well-behaved (no heavy tails)

4.4 Finding (iv): Random Adjacency with QLIKE Shows Mixed Results

GNNHAR models trained with random adjacency matrices using QLIKE loss exhibited intermediate performance.

4.4.1 Performance Comparison

From Table 3:

- GNNHAR2L (Random, QLIKE): MSE=0.008058, QLIKE=0.073542
- Outperforms: GNNHAR1L and GNNHAR3L with random adjacency
- Underperforms: Best MSE-trained models (0.008008–0.008040)
- Outperforms: Other QLIKE-trained models with true adjacency

4.4.2 Hypothesis

The combination of:

1. Random adjacency (acting as regularization)
2. QLIKE loss (different optimization landscape)

creates a unique training dynamic that partially mitigates QLIKE’s optimization difficulties while benefiting from random adjacency’s regularization effect.

4.4.3 Resolution of Open Question

We subsequently conducted experiments with **MSE loss + Random adjacency** for GNNHAR models. The results (Table 3) confirmed our hypothesis: GNNHAR2L (Random, MSE) achieved the **best performance across all experiments** with MSE = 0.007997 and QLIKE = 0.073243, representing a 1.34% improvement over HAR baseline.

This finding suggests that the combination of:

1. Random adjacency (implicit regularization)
2. MSE loss (stable optimization landscape)
3. Nonlinear GNN layers (expressive power)

creates an optimal configuration for volatility forecasting in this simulation setting.

4.5 Finding (v): Parameter Gain vs Information Gain Dilemma

The superior performance of GHAR and GNNHAR models with random adjacency matrices raises a fundamental epistemological question: **Does the improvement stem from incorporating genuine network structure information, or merely from increasing the number of model parameters?**

4.5.1 The Core Question

Consider the parameter structure:

HAR Model:

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1}\boldsymbol{\beta} \quad (18)$$

Parameters: $\boldsymbol{\beta} \in \mathbb{R}^3$ (daily, weekly, monthly)

GHAR Model:

$$\mathbb{E}(\mathbf{v}_t \mid \mathcal{F}_{t-1}) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1}\boldsymbol{\beta} + \mathbf{W}\mathbf{V}_{:t-1}\boldsymbol{\gamma} \quad (19)$$

Parameters: $\boldsymbol{\beta}, \boldsymbol{\gamma} \in \mathbb{R}^3$ (6 parameters total)

The GHAR model has **twice as many parameters** as HAR. Is the improvement due to:

1. **Information Gain Hypothesis:** The adjacency matrix $\mathbf{W}$ encodes true spillover relationships
2. **Parameter Gain Hypothesis:** The additional parameters $\boldsymbol{\gamma}$ provide more modeling flexibility

4.5.2 Evidence Supporting Parameter Gain Hypothesis

The experimental results strongly favor the **Parameter Gain Hypothesis**:

Observation 1: Random Adjacency Outperforms True Adjacency From Table 3:

- GHAR (Random): MSE = 0.008034 (0.88% improvement)
- GHAR (True): MSE = 0.008092 (0.17% improvement)
- **Ratio:** Random achieves 5.2× better improvement than true adjacency

If network structure information were the primary driver, true adjacency should dominate. Instead, a randomly generated matrix—containing *zero* information about actual spillovers—performs best.

Observation 2: Similar Improvement Magnitudes Across Adjacency Types All GHAR variants show similar order-of-magnitude improvements (0.08%–0.88%), suggesting that the γ parameters, not the structure of $\mathbf{W}$, drive performance.

Observation 3: Random Adjacency as Feature Engineering The term $\mathbf{W}\mathbf{V}_{:t-1}$ can be interpreted as **linear feature transformation**:

$$\text{New features} = \mathbf{W}\mathbf{V}_{:t-1} \quad (20)$$

Even when $\mathbf{W}$ is random, it creates linear combinations of original features, effectively performing automated feature engineering. The model learns optimal weights γ for these engineered features.

4.5.3 Alternative Interpretation: Implicit Regularization

Random adjacency matrices may act as **implicit regularization** mechanism:

- **True adjacency:** Highly structured, may overfit to training data’s spurious correlations
- **Random adjacency:** Introduces controlled noise, forcing the model to learn more robust patterns
- **Analogy:** Similar to dropout in deep learning or random feature selection

In small-sample regimes ($T = 600$ training observations), this regularization effect may dominate over information gain from true network structure.

4.5.4 Proposed Validation Experiments

To definitively resolve this question, future work should conduct:

Experiment 1: HAR-Extended with Equal Parameters Create an HAR-Extended model with 6 parameters but *without* adjacency matrices:

$$\mathbb{E}(\mathbf{v}_t) = \boldsymbol{\alpha} + \mathbf{V}_{:t-1}\boldsymbol{\beta} + (\mathbf{V}_{:t-1})^2\boldsymbol{\gamma} \quad (21)$$

where $(\mathbf{V}_{:t-1})^2$ denotes element-wise squaring (or other nonlinear transformations).

Prediction:

- If HAR-Extended achieves similar improvement to GHAR $\Rightarrow$ **Parameter Gain**
- If HAR-Extended performs poorly $\Rightarrow$ Network structure has special properties

Experiment 2: Parameter Scaling Analysis Test models with varying parameter counts:

Model	Parameters	Feature Construction
HAR	3	Daily, weekly, monthly
HAR-4	4	+ Quarterly
HAR-6	6	+ Quadratic terms
GHAR (Random)	6	+ Random $\mathbf{W}$
HAR-9	9	+ Cubic terms
GHAR-9 (Random)	9	+ Random $\mathbf{W} \times 3$ lags

Plot MSE vs number of parameters to determine if improvement is monotonic.

Experiment 3: Information-Theoretic Analysis Compute mutual information between features and target:

$$I(\mathbf{V}_{:t-1}; \mathbf{v}_t) = \text{Information in HAR features} \quad (22)$$

$$I(\mathbf{W}_{\text{true}}\mathbf{V}_{:t-1}; \mathbf{v}_t) = \text{Information in true adjacency features} \quad (23)$$

$$I(\mathbf{W}_{\text{random}}\mathbf{V}_{:t-1}; \mathbf{v}_t) = \text{Information in random adjacency features} \quad (24)$$

If $I(\mathbf{W}_{\text{true}}\mathbf{V}_{:t-1}; \mathbf{v}_t) \approx I(\mathbf{W}_{\text{random}}\mathbf{V}_{:t-1}; \mathbf{v}_t)$, this confirms parameter gain dominates.

4.5.5 Implications for Model Design

This finding has profound implications:

- ✗ **Interpretability Loss:** If random adjacency is optimal, we lose the ability to interpret $\mathbf{W}$ as economic spillover relationships
- ✓ **Practical Performance:** For pure forecasting, network estimation may be unnecessary
- ! **Causal vs Predictive:** Highlights the fundamental tension between:
 - *Causal models* (true $\mathbf{W}$, interpretable, policy-relevant)
 - *Predictive models* (optimal $\mathbf{W}$, better forecasts, black-box)

4.5.6 Theoretical Connection to Causal Inference

This phenomenon connects to the broader question in causal inference: **When does the optimal predictor require causal structure?**

In general, the optimal predictor requires causal structure only when:

$$\mathbf{W}^* = \arg \min_{\mathbf{W}} \mathbb{E}[(\mathbf{v}_t - f(\mathbf{W}, \mathbf{V}_{:t-1}))^2] = \mathbf{W}_{\text{true}} \quad (25)$$

Our results suggest $\mathbf{W}^* \neq \mathbf{W}_{\text{true}}$ in this setting, possibly because:

1. Small sample size favors regularized solutions
2. Model misspecification (linear GHAR vs nonlinear true DGP)
3. Predictive relationships differ from causal relationships

4.6 Finding (vi): High Variance Across Training Runs

The most concerning finding for practical deployment is the substantial variance in model performance across training runs with identical configurations.

4.6.1 Evidence of Instability

Best Model Architecture Changed:

- Run 1: GNNHAR3L (True, MSE) is best
- Run 2: GNNHAR2L (True, MSE) is best

Individual Model Variance:

- GNNHAR3L (True, MSE): 0.008040 (Run 1) $\rightarrow$ 0.008138 (Run 2)
- GNNHAR2L (True, MSE): 0.008134 (Run 1) $\rightarrow$ 0.008008 (Run 2)
- Some models switched from *increasing* error to *decreasing* error

4.6.2 Sources of Variance

1. **Random Weight Initialization:** PyTorch’s Xavier initialization introduces stochasticity:

```
nn.init.xavier_uniform_(self.weight)
```

2. **Mini-batch Sampling:** Random ordering of batches affects gradient trajectory
3. **Small Dataset:** Only 600 training samples amplify random effects
4. **Nonlinear Activations:** ReLU creates non-convex loss surfaces with many local minima
5. **Missing Random Seeds:** Code lacks explicit seed control

4.6.3 Implications for Reproducibility

This “alchemy” phenomenon raises critical concerns:

- ✗ Cannot reliably identify best architecture
- ✗ Results not reproducible without fixed seeds
- ✗ Difficult to compare with published benchmarks
- ✗ May require ensemble methods for stability

5 Conclusions and Recommendations

5.1 Summary of Key Insights

Table 4 summarizes our findings with severity assessments.

Table 4: Summary of Findings with Severity Assessment

Finding	Implication	Impact	Severity
(i) GLASSO failure	Alternative methods needed for adjacency estimation	Estimation accuracy	Medium
(ii) Random optimal	Random adjacency outperforms true adjacency across all model types	Model design	Critical
(iii) MSE superior	MSE training dominates QLIKE; contradicts literature	Training protocol	High
(iv) MSE+Random best	GNNHAR2L (Random, MSE) achieves 1.34% improvement—best overall	Model selection	High
(v) Parameter gain	Improvements may stem from parameters, not network information	Interpretability	Critical
(vi) High variance	Reproducibility crisis; catastrophic failures observed	Reliability	Critical

5.2 Immediate Recommendations

5.2.1 Fix Reproducibility Issues

Implement strict random seed control:

```
import random, numpy as np, torch

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.deterministic = True
```

5.2.2 Multiple Runs Protocol

- Run each configuration 10 times with seeds $\{1, 2, \dots, 10\}$
- Report mean $\pm$ standard deviation for all metrics
- Use statistical tests (t-test, Wilcoxon) to compare models
- Only claim superiority if $p < 0.05$

5.2.3 Validation Set for Early Stopping

- Split data: 60% train, 20% validation, 20% test
- Monitor validation loss during training
- Stop when validation loss plateaus (patience = 50 epochs)
- Select best model based on validation performance

5.3 Future Research Directions

5.3.1 Priority 1: Parameter Gain Hypothesis Testing

The most urgent research question is to rigorously test whether improvements come from parameters or information:

Model	Parameters	Description
HAR	3	Baseline
HAR-Extended-6	6	$\mathbf{V}\beta + \mathbf{V}^2\gamma$
HAR-Extended-9	9	Add cubic and interaction terms
GHAR (Random)	6	$\mathbf{V}\beta + \mathbf{W}_{\text{random}}\mathbf{V}\gamma$
GHAR (Identity)	6	$\mathbf{V}\beta + \mathbf{I}\mathbf{V}\gamma$ (duplicate features)
GHAR (Ones)	6	$\mathbf{V}\beta + \mathbf{J}\mathbf{V}\gamma$ (mean pooling)
GHAR (True)	6	$\mathbf{V}\beta + \mathbf{W}_{\text{true}}\mathbf{V}\gamma$

Experiment Design

Statistical Protocol

- Run each configuration 20 times with seeds $\{1, 2, \dots, 20\}$
- Compute mean $\pm$ 95% confidence interval for MSE and QLIKE
- Use ANOVA to test: H_0 : All 6-parameter models perform equally
- If rejected, use post-hoc tests (Tukey HSD) to identify significantly different groups
- Compute effect sizes (Cohen's d) to assess practical significance

Expected Outcomes

- If HAR-Extended-6 $\approx$ GHAR (Random) $\approx$ GHAR (Identity): **Parameter gain confirmed**
- If GHAR (True) $\gg$ all others: **Information gain validated**
- If GHAR (Random) $>$ GHAR (True): **Regularization hypothesis supported**

5.3.2 Priority 2: Adjacency Matrix Optimization

Treat $\mathbf{W}$ as learnable parameters with sparsity regularization:

$$L_{\text{total}} = L_{\text{prediction}} + \lambda \|\mathbf{W}\|_1 \quad (26)$$

Compare learned $\mathbf{W}$ with true adjacency to understand when and why they diverge.

5.3.3 Alternative Adjacency Estimation Methods

- VAR-based Granger causality
- Transfer entropy from information theory
- Attention mechanisms (self-learned edge weights)
- Bayesian network structure learning

5.3.4 Larger-Scale Simulations

- Increase T from 1000 to 10,000 observations
- Test if QLIKE advantages emerge with more data
- Vary network size ($N = 10, 20, 50$ stocks)
- Different network topologies (scale-free, small-world)

5.3.5 Loss Function Analysis

- Visualize loss landscapes for MSE vs QLIKE
- Compute gradient norms during training
- Try hybrid loss: $\alpha \cdot L_{\text{MSE}} + (1 - \alpha) \cdot L_{\text{QLIKE}}$
- Adaptive loss weighting based on training dynamics

5.3.6 Ensemble Methods

- Train 10 models with different random seeds
- Combine predictions via:
 - Simple averaging
 - Weighted averaging (by validation performance)
 - Stacking with meta-learner
- Measure variance reduction

5.3.7 Theoretical Investigation

Characterize conditions under which:

$$\mathbf{W}_{\text{optimal}} \neq \mathbf{W}_{\text{true}} \tag{27}$$

Connect to:

- Causal inference theory (adjustment sets)
- Regularization and bias-variance trade-off
- Information geometry and statistical manifolds

5.3.8 Real Data Validation

- Test on actual financial data (DJIA constituents)
- Compare with published benchmark studies
- Validate findings beyond simulation context

5.4 Study Limitations

1. **Single Data Realization:** Only one simulated dataset tested; should generate multiple datasets with varying parameters
2. **Simple Network:** Only 6 stocks with sparse connections; real financial networks are larger and denser
3. **No Real Data:** All results based on simulation; generalization to actual markets unknown
4. **Limited Hyperparameter Tuning:** Fixed $n_{\text{hid}} = 9$, $\text{lr} = 0.001$; optimal settings may differ
5. **No Statistical Testing:** Improvements of 0.1–1% may be within noise level; need confidence intervals

6 Conclusions

This simulation study reveals several unexpected and theoretically profound behaviors in the GNNHAR framework that fundamentally challenge our understanding of network-based volatility modeling:

1. **The Adjacency Matrix Paradox:** Random adjacency matrices consistently outperform true causal structure across both linear (GHAR) and nonlinear (GNNHAR) models. The best-performing model overall—GNNHAR2L (Random, MSE) with 1.34% improvement—uses a randomly generated adjacency matrix containing zero information about actual spillover relationships.
2. **Parameter Gain vs Information Gain:** The superior performance of random adjacency strongly suggests that improvements stem primarily from *additional model parameters* (γ) rather than genuine *network structure information* encoded in $\mathbf{W}$. This raises fundamental questions about whether GHAR/GNNHAR models truly capture spillover effects or merely perform automated feature engineering.
3. **Estimation Challenges:** Standard methods like GLASSO catastrophically fail to recover true network structure from finite samples with nonlinear relationships, achieving only 0.08% improvement compared to 0.88% for random adjacency.
4. **Loss Function Dynamics:** MSE training dominates QLIKE training across all configurations, contradicting existing literature. MSE-trained models outperform QLIKE-trained models even when evaluated using the QLIKE metric itself, suggesting fundamental optimization difficulties with QLIKE loss in small-sample regimes.
5. **Optimal Configuration:** The combination of (i) random adjacency, (ii) MSE loss, and (iii) 2-layer GNN architecture achieves best performance, suggesting that implicit regularization from random structure combined with stable optimization creates an effective forecasting model.
6. **Reproducibility Crisis:** High variance across training runs, including catastrophic failures (GNNHAR3L achieving $\text{MSE} > 0.1$), represents the most critical practical concern. The “alchemy” problem requires rigorous experimental protocols with fixed random seeds, multiple runs, and statistical validation.
7. **Interpretability Trade-off:** If $\mathbf{W}_{\text{optimal}} \neq \mathbf{W}_{\text{true}}$, we face an irreconcilable tension between:
 - *Causal interpretation:* Using true $\mathbf{W}$ for economic insights and policy analysis
 - *Predictive performance:* Using optimal $\mathbf{W}$ (possibly random) for accurate forecasts

6.1 Fundamental Implications

The finding that $\mathbf{W}_{\text{optimal}} \neq \mathbf{W}_{\text{true}}$ connects to deeper questions in causal inference and statistical learning theory:

- **Causal vs Predictive Modeling:** This study demonstrates that the structure minimizing prediction error may fundamentally differ from the true causal structure, especially in small-sample, potentially misspecified settings.
- **Interpretability Cost:** Achieving optimal predictive performance may require sacrificing model interpretability. For financial applications, this trade-off is particularly concerning: forecasts guide trading decisions, but understanding systemic risk requires causal knowledge.
- **Validation Strategy:** Our results suggest that validating network models solely on predictive performance may be misleading. Models should be evaluated on both (i) forecast accuracy and (ii) ability to recover true network structure in simulation studies where ground truth is known.

6.2 Recommendations for Practice

Given these findings, we recommend:

1. **Control for Parameter Count:** When comparing HAR, GHAR, and GNNHAR, ensure fair comparisons by testing HAR-Extended models with equivalent parameter counts but alternative feature engineering.

2. **Multiple Adjacency Specifications:** Report results for true, estimated (GLASSO/VAR), and random adjacency matrices to assess sensitivity to network specification.
3. **Ensemble Methods:** Given high training variance, deploy ensemble models averaging across multiple random initializations rather than single best-performing runs.
4. **Separate Objectives:** Maintain separate model pipelines for:
 - *Forecasting:* Use best-performing configuration (possibly random adjacency)
 - *Risk Analysis:* Use causally-grounded adjacency for spillover interpretation

Future work should rigorously test the parameter gain hypothesis through controlled experiments, explore alternative adjacency estimation methods, and investigate whether these findings generalize to real financial data and larger networks.

Acknowledgments

This work was conducted as part of a quantitative finance research project. Code and data are available in the GNNHAR repository.

A Data Generation Code Structure

The data generation pipeline implemented in `generate_mock_data.py` follows a hierarchical structure:

1. Generate leaf nodes (CVX, AXP, BA) as pure random walks
2. Generate 1-hop neighbors (GS, JPM) with spillovers from leaf nodes
3. Generate target node (IBM) with spillovers from 1-hop neighbors
4. Apply nonlinear transformations (RV⁴ terms)
5. Add heteroskedastic noise

B Model Training Details

B.1 Linear Models

HAR and GHAR models were estimated using ordinary least squares:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{v} \quad (28)$$

where $\mathbf{X}$ contains HAR features (and graph-aggregated features for GHAR).

B.2 Neural Network Models

GNNHAR models were trained using the Adam optimizer with the following update rule:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (29)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (30)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (31)$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

C Computational Environment

- **Operating System:** Windows 10
- **Python Version:** 3.x
- **PyTorch Version:** Latest stable release
- **Hardware:** CPU-based training (sufficient for this scale)
- **Training Time:** ≈ 5 minutes per GNNHAR model

D Complete Results Tables

Tables 1–3 in the main text present the comprehensive experimental results. Visual summaries are available in:

- GNNHAR/data/mock/GNN-result1.png (First run results)
- GNNHAR/data/mock/GNN-result2.png (Second run results)